

Task1.py

Я створив функції `print_card()` та `print_card_args()`. Перша функція використовує дані безпосередньо з програми, а друга отримує ім'я, прізвище, групу та рік народження через параметри. Також я перевіряв позиційні, іменовані та змішані аргументи, а також значення параметра `group` за замовчуванням. Додатково було розглянуто помилки, які виникають при неправильному виклику функції.

```
6
7 def print_card():
8     """Print student's personal information."""
9     print(f"Name: {name} {surname}")
10    print(f"Group: {group}")
11    print(f"Birth year: {year}")
12
13
14 def print_card_args(name, surname, group="IT-31", year=2008):
15     """Print student's information received through parameters."""
16     print(f"{name} {surname}, {group}, {year}")
17
18
19 print(f"{name} {surname}, {group}")
20
21 print("--- no parameters, call 1 ---")
22 print_card()
23
24 print("--- no parameters, call 2 ---")
25 print_card()
26
27 print("--- no parameters, call 3 ---")
28 print_card()
29
30 print("--- positional arguments ---")
31 print_card_args("Illya", "Vorobey", "IT-31", 2008)
32
33 print("--- keyword arguments ---")
34 print_card_args(year=2008, group="IT-31", surname="Vorobey", name="Illya")
35
36 print("--- mixed arguments ---")
37 print_card_args("Illya", "Vorobey", group="IT-31", year=2008)
38
39 print("--- default group ---")
40 print_card_args("Illya", "Vorobey", year=2008)
41
42
```

Task2.py

Я порівняв функції, які використовують `print` і `return`. Функція `print_age()` лише виводить результат і повертає `None`, тоді як `get_age()` повертає число, яке можна використовувати в подальших обчисленнях. Також я реалізував перевірку некоректного року народження та ранній вихід із функції за допомогою `return`.

```

task2.py > ...
4  y = 2008
5
6
7  def print_age(year):
8      """Print the age calculated from the birth year."""
9      age = 2026 - year
10     print(f"Age: {age}")
11
12
13  def get_age(year, current_year=2026):
14      """Return the age calculated from the birth year."""
15      if year > current_year or year < 0:
16          return -1
17
18      return current_year - year
19
20     print("after return")
21
22     (function) def print_age(year: Any) -> None
23     Print the age calculated from the birth year.
24
25     print_age(y)
26
27     print(f"print_age returned: {print_age(y)}")
28
29     age = get_age(y)
30
31     print(f"Age from get_age: {age}")
32     print(f"Age in months: {age * 12}")
33     print(f"Age in weeks: {age * 52}")
34     print(f"Age in 2030: {get_age(y, 2030)}")
35
36     print(f"Invalid year 3000 gives: {get_age(3000)}")
37
38     print_age(y) * 12
39

```

Task3.py

Я створив набір функцій для роботи з моїм прізвищем Vorobey: визначення ініціалів, підрахунок символів і голосних та перевертання рядка. Усі функції мають анотації типів, а функція count_letters() також містить docstring. Під час виконання я перевіряв значення за замовчуванням, іменовані аргументи, а також отримав документацію та анотації функції через спеціальні атрибути.

```

task3.py > ...
1  name = "Illya"
2  surname = "Vorobey"
3
4
5  def get_initials(name: str, surname: str) -> str:
6      """Return initials from name and surname."""
7      return f"{name[0]}.{surname[0]}."
8
9
10 def count_letters(text: str, letter: str = "a") -> int:
11     """Return how many times letter occurs in text."""
12     count = 0
13
14     for char in text:
15         if char.lower() == letter.lower():
16             count += 1
17
18     return count
19
20
21 def count_vowels(text: str) -> int:
22     """Return the number of vowels in text."""
23     vowels = "aeiouy"
24     count = 0
25
26     for char in text.lower():
27         if char in vowels:
28             count += 1
29
30     return count
31
32
33 def reverse_text(text: str) -> str:
34     """Return text in reverse order without slicing."""
35     result = ""
36
37
38
39
40
41
42
43 print(f"{name} {surname}")
44 print(f"Initials: {get_initials(name, surname)}")
45
46 c = len(surname)
47 vowels = count_vowels(surname)
48 consonants = count_letters(surname) + count_letters(surname, "e") + count_letters(surname, "i")
49
50 print(f"Letters in surname: {c}")
51 print(f"Vowels: {vowels}, consonants: {c - vowels}")
52
53 for vowel in "aeiouy":
54     print(f"{vowel}: {count_letters(surname, letter=vowel)}")
55
56 print(f"Default letter 'a': {count_letters(surname)}")
57 print(f"Reversed surname: {reverse_text(surname)}")
58
59 print(f"Docstring: {count_letters.__doc__}")
60 print(f"Annotations: {count_letters.__annotations__}")
61

```

Task4.py

Я розділив програму для роботи з оцінками на окремі функції. Кожна функція виконує одну конкретну дію: введення оцінки, визначення буквеного результату, обчислення середнього, підрахунок оцінок вище середнього або виведення звіту. Для моїх п'яти оцінок 47, 86, 97, 89, 90 середнє значення становить 81.80, найкращий бал — 97, найгірший — 47, а кількість оцінок вище середнього — 3.

```
task4.py > ...
1  def read_grade(prompt):
2      """Read and return an integer grade from 0 to 100."""
3      while True:
4          value = input(prompt)
5
6          if not value.isdigit():
7              print("Error: digits only")
8              continue
9
10         grade = int(value)
11
12         if grade < 0 or grade > 100:
13             print("Error: the value must be between 0 and 100")
14             continue
15
16         return grade
17
18
19  def to_letter(grade):
20      """Convert a numeric grade to a letter grade."""
21      if grade >= 90:
22          return "A"
23      if grade >= 82:
24          return "B"
25      if grade >= 74:
26          return "C"
27      if grade >= 64:
28          return "D"
29      if grade >= 60:
30          return "E"
31      return "F"
32
33
34  def average(grades):
35      """Return the arithmetic mean of grades."""
36      return sum(grades) / len(grades)
37
```

```

39 def count_above(grades, limit):
44     if grade > limit:
45         count += 1
46
47     return count
48
49
50 def print_report(name, group, grades):
51     """Print a report containing the student's grades and statistics."""
52     avg = average(grades)
53
54     print("--- Report ---")
55     print(f"Student: {name}, group {group}")
56     print("Grades:", *grades)
57     print(f"Average: {avg:.2f} -> {to_letter(avg)}")
58     print(f"Best: {max(grades)}, worst: {min(grades)}")
59     print(f"Above average: {count_above(grades, avg)}")
60
61
62 def main():
63     """Read five grades and print the student's report."""
64     name = "Illya Vorobey"
65     group = "IT-31"
66     n = len("Illya")
67
68     print(f"{name}, {group}")
69
70     grades = []
71
72     for i in range(n):
73         grade = read_grade(f"Grade {i + 1} (0-100): ")
74         grades.append(grade)
75
76     print_report(name, group, grades)
77
78

```